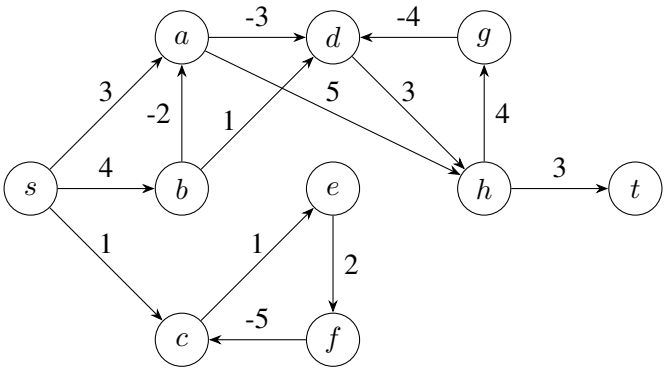
Problem Set 4

CMSC 27200: Theory of Algorithms

Problem 1: Bellman-Ford (25 points)

(a) 20 points: Simulate the execution of the Bellman-Ford algorithm on the graph below. For your answer, for each $k \in \{0, 1, 2, \ldots, 10\}$ and each vertex $v \in V(G)$, you should provide the length of the shortest path from s to v which uses at most k edges (as a table). You should also find the shortest path from s to t from your table.



Solution:

k	s	a	b	c	d	e	f	g	h	t
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞
1	0	3	4	1	∞	∞	∞	∞	∞	∞
2	0	2	4	1	0	2	∞	∞	8	∞
3	0	2	4	1	-1	2	4	12	3	11
4	0	2	4	-1	-1	2	4	7	2	6
5	0	2	4	-1	-1	0	4	6	2	5
6	0	2	4	-1	-1	0	2	6	2	5
7	0	2	4	-3	-1	0	2	6	2	5
8	0	2	4	-3	-1	-2	2	6	2	5
9	0	2	4	-3	-1	-2	0	6	2	5
10	0	2	4	-5	-1	-2	0	6	2	5

Shortest path from s to t: $s \to b \to a \to d \to h \to t$

(b) 5 points: For this graph, for which vertices $v \in V(G)$ does Bellman-Ford correctly compute the length of the shortest path from s to v? What causes Bellman-Ford to fail for the other vertices? If we look at the 10th iteration, is there a warning sign for this failure?
Solution: Bellman-Ford correctly computes the length of the shortest path from s to a, b, d, g, h and t. Bellman-Ford fails on vertices c, e and f because they lie on a negative cycle with weight $1+2-5=-2$. The warning sign is that a value is updated in the 10th iteration despite the fact that the longest path can use maximum $n-1$ edges (or n vertices).

Problem 2: Music Festival (25 points)

You are attending a large music festival. At this festival, there are k venues and n time slots. If you are at venue i during time slot j then you will derive enjoyment h_{ij} from the show there. Unfortunately, the venues are rather far apart, so while you can change venues during the festival, each time you move from one venue to another takes one time slot. Given this, what is the maximum total enjoyment you can have?

We can state this problem mathematically as follows: Given a $k \times n$ matrix H where h_{ij} is your enjoyment from being at venue i during time slot j, find a schedule $S = ((i_1, j_i), \ldots, (i_n, j_n))$ such that

1. $1 \leq j_1 < j_2 < \ldots < j_l \leq n$ (you can be at most one venue at any given time slot)
2. For all $t \in [n-1]$, if $i_{t+1} \neq i_t$ then $j_{t+1} \geq j_t + 2$ (traveling between venues takes one time slot)
3. $\sum_{t=1}^{n} h_{i_t j_t}$ is maximized.

Give a polynomial time algorithm to solve this problem, prove that your algorithm is correct, and analyze the runtime of your algorithm.
Solution: $DP[i,j]$ = maximum enjoyment achievable at venue i by the end of time slot j

Algorithm 1 Maximum Total Enjoyment at a Music Festival

Input: $k \times n$ matrix H
for $i = 1, \ldots, k$ **do**
    $DP[i,1] \leftarrow h_{i1}$
end for
for $j = 1, \ldots, n$ **do**
    for $i = 1, \ldots, k$ **do**
        $DP[i,j] \leftarrow h_{ij} + \max_{i' \in [k]} \begin{cases} DP[i, j-1], & \text{if } i' = i \\ DP[i', j-2], & \text{otherwise} \end{cases}$
    end for
end for
return $\max_{i \in [k]}(DP[i,n])$

Correctness:

Base case: At time slot 1, $DP[i,1] = h_{i1}$ for all $i \in [k]$, so algorithm 1 returns $\max_{i\in[k]}(DP[i,1]) = \max_{i\in[k]}(h_{i1})$.

Recursive step: The maximum enjoyment achievable at venue i by the end of time slot j is the enjoyment derived from that show, h_{ij}, plus the maximum enjoyment derived from shows before j. This could be achieved either by staying at venue i with an enjoyment of $DP[i, j-1]$, or by traveling from a previous venue i' with an enjoyment of $DP[i', j-2]$ because it takes an additional time slot to travel from i' to i.

Runtime: We have to compute kn entries of DP, and each computation considers k previous entries, plus a final maximum over k. This gives a total runtime of $O(k^2 n)$.

Problem 3: Trading Stocks (25 points)

You are looking at a given stock over n consecutive days, where the price for day $i \in \{1, \cdots, n\}$ is p_i per share (assume for simplicity that the price is fixed during each day). You would like to know: how should you choose a day i on which to buy the stock and a later day $j > i$ on which to sell it, so that the profit per share, $p_j - p_i$, is maximized?

You may assume for simplicity that $p_1 < p_2$, so that you can always make money. Give an algorithm to find the optimal numbers i and j in $O(n)$ time. You need to show that your algorithm is correct, and analyze its runtime.

Solution:

Algorithm 2 Maximum Profit from Trading Stocks

Input: prices $p_1, \ldots, p_n$
Output: days buy and $sell$

```
minPrice ← p_1
minDay ← 1
maxProfit ← p_2 − p_1
buy ← 1
sell ← 2
for i = 3, ..., n do
    if p_i − minPrice > maxProfit then
        maxProfit ← p_i − minPrice
        buy ← minDay
        sell ← i
    end if
    if p_i < minPrice then
        minPrice, minDay ← p_i, i
    end if
end for
return (buy, sell)
```

Correctness: At the beginning of the iteration corresponding to day i, the $maxProfit$ is the maximum profit achievable using days $1, \ldots, i-1$ and $minPrice$ is the minimum price among days $1, \ldots, i-1$. Before the first iteration, the maximum profit is achievable by buying on day 1 and selling on day 2 by the assumption, and $minPrice$ is the price on day 1. On day i, $maxProfit$ is either the $maxProfit$ from day $i-1$ or the profit from buying on the day with $minPrice$ and selling on day i, whichever is greater. Similarly, $minPrice$ is either the $minPrice$ from day $i-1$ or p_i, whichever is lower. After the last iteration (corresponding to day n), buy and $sell$ will represent the days on which to trade in order to achieve $maxProfit$, so algorithm 2 will find the optimal days to buy and sell on.

Runtime: Each of the $O(n)$ iterations takes $O(1)$ time, for a total runtime of $O(n)$.

Problem 4: Unhidden Points (25 points)

You are given n points $\{(x_i, y_i)\}_{i \in [n]}$ on the plane. Assume for simplicity that the coordinates $x_1, \cdots, x_n$ and $y_1, \cdots, y_n$ are all distinct. We say that point j *hides* point i if $x_i < x_j$ and $y_i < y_j$, i.e., both coordinates of point j are larger than point i.

Give an $O(n \log n)$ time algorithm to find the set of points that are not hidden by any other points. In other words, find the set $S = \{i : \text{For all } j \in [n] \setminus \{i\}, x_i > x_j \text{ or } y_i > y_j\}$. Explain why your algorithm is correct and takes $O(n \log n)$ time.

Hint: What algorithmic paradigm do you want to use for this problem? If $i_1, \ldots, i_m$ are the unhidden points and $x_{i_1} < \ldots < x_{i_m}$, what can we say about $y_{i_1}, \ldots, y_{i_m}$?
Solution:

Algorithm 3 Maximum Set of Unhidden Points

Input: points $\{(x_i, y_i)\}_{i \in [n]}$
Output: set S

```
Sort points by decreasing x
add 1 to S
Y_max ← y_1
for i = 2, ..., n do
    if y_i > Y_max then
        add i to S
        Y_max ← y_i
    end if
end for
return S
```

Correctness: At the beginning of the iteration corresponding to point i, $Y_{\max}$ is the maximum y for any point with $x > x_i$. Before the first iteration, the maximum y is trivially y_1. Up to point i, y_i is either greater or less than $Y_{\max}$ since all y values are distinct by assumption. If y_i is greater than

$Y_{\max}$, $x_i < x_j$ and $y_i > y_j$, so point i is not hidden by any of the points already considered. It also cannot be hidden by any future points because it has a greater x than them. Thus, point i is added to set S and $Y_{\max}$ is updated. If y_i is less than $Y_{\max}$, it is hidden by the point with $y = Y_{\max}$ and $x > x_i$, so it is not added to S. After the last iteration, all of the n points will have been considered, and set S will contain only the unhidden points.

Runtime: Each of the $O(n)$ iterations takes $O(1)$ time and the sort takes $O(n \log n)$ time, for a total runtime of $O(n \log n)$.